

Autoencodeurs embarqués sur ESP32 pour la détection de vibrations

Alexandre Couté¹, Ana Fernanda Quaresma², Isadora Ferrão¹, Kalinka Branco², Catherine Dezan¹

¹ Université de Bretagne Occidentale, Lab-STICC

² Universidade de São Paulo, São Carlos

alexandre.coute@outlook.fr

Résumé

L'Industrie 4.0 déploie des modèles d'IA pour prédire les défaillances par analyse vibratoire. Cet article évalue deux modèles d'autoencodeur non-supervisé (Autoencodeur standard et VAE) pour la détection de vibration. Ces modèles sont évalués sur le Dataset CWRU et comparés avec un modèle supervisé de type Random Forest. Nous comparons leurs contraintes d'apprentissage, leur capacité de classification et leur portabilité sur ESP32, menant à la proposition d'une architecture optimisée pour la sûreté de fonctionnement.

Mots-clés

Edge AI, ESP32, Maintenance prédictive, Random Forest, Autoencodeur, TinyML, CWRU Dataset

Abstract

Industry 4.0 deploys AI models to predict failures through vibration analysis. This paper evaluates two unsupervised autoencoder models (standard Autoencoder and VAE) for vibration detection. These models are evaluated on the CWRU dataset and compared with a supervised Random Forest model. We compare their training constraints, classification ability, and portability on ESP32, leading to the proposal of an architecture optimized for functional safety.

1 Introduction et méthodologie

La maintenance prédictive industrielle requiert une analyse locale (Edge Computing) pour limiter la latence et sécuriser les données. Dans cette étude, on propose d'utiliser deux modèles non supervisés (Autoencodeur classique et Autoencodeur Variationnel) et de les comparer à un modèle supervisé (Random Forest).

Les données, issues de la Case Western Reserve University (CWRU) [1], ont été transformées par FFT (128 bandes) et normalisées. Afin de garantir la fiabilité statistique des modèles, l'entraînement utilise 80% des données disponibles, tandis que les 20% restants sont réservés à la validation et à l'évaluation des performances.

Pour garantir une comparaison non biaisée de leur capacité de séparation, l'étude se concentre sur l'état sain et trois défauts à une **sévérité unique de 0,007 pouce**. Ces défauts correspondent à des trous de 0,18 mm percé sur la machine

pour tester la capacité de l'IA à détecter l'apparition de premiers stades d'usure. Une méthode de *Data Augmentation* par injection de bruit gaussien permettra d'évaluer la robustesse de l'extraction des caractéristiques.

2 Description des modèles

Dans cette étude, trois modèles de type machine learning sont évalués :

- **Random Forest (RF)** : Un modèle d'apprentissage supervisé basé sur un ensemble d'arbres de décision. Sa structure logique le rend très léger, mais il requiert des données annotées pour l'ensemble des comportements (sains et défectueux).
- **Autoencodeur (AE) standard** : Un réseau de neurones non supervisé conçu pour compresser les données d'entrée dans un espace latent réduit, puis les reconstruire. L'objectif du modèle est de minimiser l'erreur quadratique moyenne (MSE) entre le signal d'entrée et le signal reconstruit [3].
- **Autoencodeur Variationnel (VAE)** : Une variante de l'autoencodeur qui cartographie les entrées selon une distribution probabiliste, permettant un espace latent plus continu mais avec une régularisation plus contraignante [2].

3 TinyML pour l'embarqué

La mise en oeuvre du modèle d'autoencodeur sur le microcontrôleur ESP32 (~520 Ko de RAM) nécessite un processus d'exportation et de conversion spécifique. Les modèles Keras traditionnels étant trop lourds pour l'ESP32, le modèle est compressé à l'aide de **TFLiteConverter** vers le format TensorFlow Lite, qui est léger et optimisé pour l'inférence. Afin d'éviter l'utilisation d'une carte SD externe, ce fichier binaire est traduit en un tableau C. Ce tableau est ainsi flashé directement dans la mémoire du microcontrôleur.

Dans l'environnement Arduino IDE, l'exploitation du modèle requiert l'installation de la bibliothèque **TFLite_ESP32**. L'architecture logicielle alloue manuellement un bloc de mémoire fixe dédié aux calculs de l'IA, appelé "Tensor Arena". Par ailleurs, les paramètres statistiques comme les minimums, maximums, et le seuil d'alerte calculé lors de l'entraînement sont intégrés comme

constantes dans le programme.

La boucle d'exécution logicielle s'organise en trois étapes :

- Le système capture des fenêtres de 256 points du signal et les transmet à l'entrée du modèle. L'autoencodeur passe les points dans son réseau de neurones pour générer le signal reconstruit.
- Le programme calcule ensuite en C++ la MSE mesurant la différence entre l'entrée et la reconstruction.
- Cette erreur est comparée au seuil d'alerte. Si la valeur calculée est excessivement haute par rapport au seuil, une anomalie est détectée, sinon l'état est considéré normal.

4 Expérimentations et résultats

L'évaluation repose sur l'accuracy, la classification (dont la classification forcée des signaux inconnus), la PCA, l'analyse des fréquences et l'empreinte matérielle. Le Tableau 1 synthétise ces performances.

TABLE 1 – Performances et contraintes matérielles des modèles

Critère	Random Forest	AE Standard	VAE
Accuracy (Sain + 0.007")	99,94%	88,5%	97,5%
Risque de classification forcée	100%	0%	0%
Temps Inférence (théorique)	< 1 ms	~ 30 ms	~ 50 ms
Mémoire RAM (ESP32)	< 1 Ko	~ 40 Ko	~ 60 Ko

Avec une **accuracy** de 99,94%, le **RF** domine la classification supervisée en isolant les pannes connues. L'**AE** (88,5%) et le **VAE** (97,5%) affichent une précision globale légèrement moindre car leur objectif n'étant pas de séparer des classes mais de modéliser le signal sain, ils sacrifient une fraction d'**accuracy** au profit d'une sécurité indispensable face à l'imprévu.

En milieu industriel, posséder des données étiquetées pour chaque dégradation est souvent irréaliste. C'est la vulnérabilité du **RF** (supervisé) : exigeant un dataset complet, il assigne tout signal inédit à une classe connue, générant un diagnostic erroné (**classification forcée**). À l'inverse, les **autoencodeurs** (non supervisés) s'entraînent uniquement sur l'état sain pour modéliser la norme. Tout signal déviant fait alors exploser la MSE, alertant l'opérateur. Cette flexibilité garantit un système robuste.

La **PCA** (Fig. 1) vérifie si le modèle structure logiquement les données en regroupant les états mécaniques similaires. Pour le **RF**, dépourvu d'espace latent, la projection des entrées brutes montre un chevauchement des pannes. L'**AE** projette efficacement les signaux en clusters géométriques isolés. Le **VAE** regroupe également les classes, mais sa contrainte probabiliste force un léger chevauchement.

L'analyse des **fréquences** (Fig. 2) révèle que **RF** base ses décisions en ciblant uniquement l'importance de pics fréquentiels précis du spectre FFT. À l'inverse, l'**AE** et le **VAE** analysent le spectre dans sa globalité pour reconstruire la

signature du signal sain. Face à une anomalie, c'est l'écart entre le spectre d'entrée et la reconstruction qui déclenche l'alerte.

La compression de l'information est le pilier de l'**embarcation sur ESP32**, rendant l'inférence des **autoencodeurs** gérable. Le **RF**, quant à lui, ne compresse pas les données mais compense par une conversion en règles C++ natives. Il offre ainsi une empreinte quasi nulle (< 1 Ko) et une latence minimale.

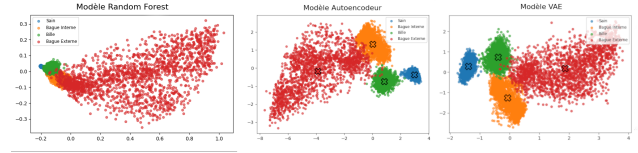


FIGURE 1 – Projection PCA des trois modèles.

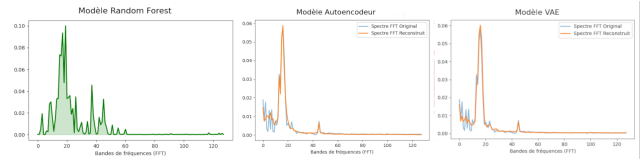


FIGURE 2 – Fréquences des trois modèles et reconstructions des autoencodeurs.

Le bilan de cette étude souligne le compromis entre **accuracy** et **généralisation**. Si le RF offre une exactitude sur les pannes apprises, les **autoencodeurs** délimitent strictement la normalité pour isoler toute défaillance inédite. C'est cette complémentarité qui justifie notre perspective d'architecture hybride : l'autoencodeur assure une détection universelle, tandis que le **RF** spécialise le diagnostic.

5 Conclusion et perspectives

Ce travail confirme la viabilité de l'IA embarquée pour la maintenance prédictive sur architectures contraintes. En perspective, nous proposons une solution hybride rapide : l'autoencodeur servira de détecteur universel, relayé par le Random Forest pour classifier les défauts. Afin d'éviter toute classification forcée, un seuil de confiance de 80% isolera les pannes inconnues, garantissant ainsi un diagnostic sûr et exhaustif en conditions réelles.

Références

- [1] CWRU Bearing Data Center. Case Western Reserve University Dataset, <https://engineering.case.edu/bearingdatacenter>, 2026.
- [2] R. Ibrahim, et al. Fault Detection Based on Vibration Measurements and VAE. *IEEE OJIA*, 2024.
- [3] W. Cheng, et al. Deep Robust Autoencoder based Framework for Bearing Fault Detection. *IEEE PHM*, 2022.